

Distribuidora de Bebidas

Guia de Configuração e Testes

Este guia foi feito para quem nunca mexeu no projeto antes.

Siga os passos na ordem e tudo vai funcionar!

Parte 1 — O que você precisa instalar

1.1 Java 21

O Java é necessário para rodar os serviços do projeto.

1. Acesse: <https://www.microsoft.com/openjdk>
2. Clique em Download e escolha JDK 21 — Windows x64 .msi
3. Instale normalmente (next, next, finish)
4. Para verificar se funcionou, abra o PowerShell e digite:

```
java -version
```

Deve aparecer algo como: openjdk version "21.x.x"

1.2 MySQL

O MySQL é o banco de dados do projeto.

5. Acesse: <https://dev.mysql.com/downloads/installer/>
6. Baixe o MySQL Installer for Windows
7. Durante a instalação, escolha Developer Default
8. Na etapa de senha, defina uma senha para o usuário root — anote essa senha, você vai precisar dela!
9. O instalador também vai instalar o MySQL Workbench — é a ferramenta visual para gerenciar o banco

1.3 Docker Desktop

O Docker é necessário para rodar o Kafka (sistema de mensagens entre os serviços).

10. Acesse: <https://www.docker.com/products/docker-desktop/>
11. Clique em Download for Windows
12. Instale normalmente
13. Após instalar, abra o Docker Desktop antes de usar — ele precisa estar rodando em segundo plano
14. Quando estiver pronto, vai aparecer o ícone de uma baleia 🐳 na barra de tarefas do Windows

1.4 IntelliJ IDEA Community

É a IDE onde você vai abrir e rodar o projeto.

15. Acesse: <https://www.jetbrains.com/idea/download/>
16. Role a página até encontrar a versão Community — gratuita

17. Clique em Download
18. Durante a instalação, marque as opções:
 - Add "bin" folder to the PATH
 - .java (associar arquivos Java)
19. Instale e reinicie o computador se solicitado

Parte 2 — Clonar o projeto pelo IntelliJ

Você vai baixar o projeto diretamente pelo IntelliJ, sem precisar do terminal.

20. Abra o IntelliJ IDEA
21. Na tela inicial, clique em Get from VCS
22. No campo URL, cole:

```
https://github.com/JoaoPedroLopesOliveira/distribuidora_de_bebidas.git
```

23. No campo Directory, escolha onde quer salvar o projeto (ex:
C:\Users\SeuNome\Documents\projetos)
24. Clique em Clone
25. Aguarde o IntelliJ baixar o projeto

Parte 3 — Criar sua branch de testes

Antes de qualquer coisa, você precisa criar uma branch separada para trabalhar. Isso é importante para não mexer no código principal da main.

 A main é um código estável. A branch testes_SEU-NOME é onde você trabalha com segurança.

No IntelliJ, abra o Terminal (barra inferior da tela) e rode:

```
git checkout -b testes_SEU-NOME    → cria a branch e entra nela

# (faz as alterações no código, testes etc)

git add .                          → prepara todos os arquivos modificados
git commit -m "descrição"          → salva localmente
git push origin estoque             → sobe a branch pro GitHub

# --- para atualizar a main ---
git checkout main                  → volta pra branch principal
git merge estoque                  → traz as mudanças da estoque pra main
git push origin main               → atualiza a main no GitHub
```

Substitua SEU_NOME pelo seu nome (ex: testes-joao).

Para confirmar em qual branch você está:

```
git branch
```

A branch com * na frente é a que está ativa. Deve aparecer * testes_SEU_NOME.

Parte 4 — Criar os bancos de dados

O projeto usa três bancos de dados separados. Você precisa criá-los na sua máquina.

26. Abra o MySQL Workbench
27. Clique na instância local para conectar (geralmente chamada de Local instance MySQL)
28. Digite sua senha do root
29. No topo, clique no ícone de novo script SQL
30. Cole e execute o seguinte:

```
CREATE DATABASE db_distribuidora_vendas;  
CREATE DATABASE db_distribuidora_compras;  
CREATE DATABASE db_distribuidora_estoque;
```

31. Clique no raio ⚡ para executar
32. Os três bancos vão aparecer no painel esquerdo em Schemas

 As tabelas dentro de cada banco são criadas automaticamente quando você rodar os serviços — não precisa criar na mão!

Parte 5 — Ajustar a senha do banco nos arquivos de configuração

Cada serviço tem um arquivo de configuração onde você precisa colocar a senha do seu MySQL. (Recomenda-se deixar usuário e senha como root)

Serviço de Compras

No IntelliJ, navegue até:

```
compras → src → main → resources → application.properties
```

Encontre essa linha e troque sua_*senha* pela sua senha do MySQL:

```
spring.datasource.password=sua_senha
```

Serviço de Vendas

Navegue até:

```
vendas → src → main → resources → application.properties
```

Faça a mesma coisa:

```
spring.datasource.password=sua_senha
```

Serviço de Estoque

Navegue até:

```
estoque → src → main → resources → application.yml
```

Encontre e ajuste:

```
spring:
  datasource:
    password: sua_senha
```

 **Atenção:** Não faça commit dessas alterações de senha! São configurações locais da sua máquina.

Parte 6 — Subir o Kafka com Docker

O Kafka é o sistema que permite os serviços se comunicarem. Ele roda via Docker.

33. Certifique-se que o Docker Desktop está aberto e com o ícone da baleia na barra de tarefas
34. No IntelliJ, clique em Terminal na barra inferior
35. Navegue até a raiz do projeto:

```
cd distribuidora_de_bebidas
```

36. Rode o comando:

```
docker-compose up -d
```

37. Aguarde baixar as imagens (pode demorar na primeira vez)
38. Para confirmar que está rodando:

```
docker ps
```

Deve aparecer dois containers: kafka e zookeeper com status Up.

Parte 7 — Abrir cada serviço no IntelliJ

O projeto tem três serviços separados. Cada um precisa ser aberto como projeto Maven no IntelliJ.

39. No IntelliJ, vá em File → Open
40. Navegue até a pasta do projeto e entre na pasta compras
41. Selecione o arquivo pom.xml
42. Clique em Open as Project
43. Aguarde o Maven baixar as dependências (barra de progresso no canto inferior direito)
44. Repita os mesmos passos para as pastas vendas e estoque

Você pode ter os três abertos ao mesmo tempo em janelas diferentes do IntelliJ.

Parte 8 — Rodar os três serviços

Cada serviço precisa estar rodando para os testes funcionarem.

Em cada projeto aberto no IntelliJ:

45. No painel esquerdo, navegue até:

```
src → main → java → com.distribuidora.[serviço] → [Nome]Application.java
```


46. Clique no ícone de play verde ao lado do método main
47. Aguarde aparecer no console:

```
Started [Nome]Application in X seconds
```

Portas de cada serviço:

Serviço	Arquivo	Porta
Estoque	EstoqueApplication.java	8081
Compras	ComprasApplication.java	8082
Vendas	VendasApplication.java	8083

 **Atenção:** Os três precisam estar rodando ao mesmo tempo!

Parte 9 — Testes unitários

Onde fica a pasta test

No IntelliJ, dentro de cada serviço:

```
src
├── test
│   └── java
│       └── com.distribuidora.[serviço] ← crie seus testes aqui
```

Se a pasta test não aparecer no painel, clique com o botão direito nela → Mark Directory as → Test Sources Root.

Exemplos de testes para implementar

Testes de Service — testam a lógica de negócio isolada

- Verificar se ao criar uma compra o status começa como PENDENTE
- Verificar se ao criar uma venda a data é preenchida automaticamente
- Verificar se o Producer Kafka é chamado ao registrar uma venda
- Verificar se o Producer Kafka é chamado ao registrar uma compra
- Verificar se ao deletar um item o repository é chamado corretamente
- Verificar se ao listar todos os itens o repository é chamado corretamente

Testes de Repository — testam a comunicação com o banco

- Verificar se um produto salvo pode ser buscado por ID
- Verificar se a listagem retorna todos os itens cadastrados
- Verificar se um item deletado não aparece mais na listagem

Testes de Controller — testam os endpoints HTTP

- Verificar se o endpoint POST /api/vendas retorna status 201
- Verificar se o endpoint GET /api/estoque retorna status 200
- Verificar se o endpoint GET /api/compras/{id} retorna 404 para ID inexistente
- Verificar se o endpoint DELETE /api/vendas/{id} retorna 204

Testes de Validação — testam regras de negócio

- Verificar se uma venda com quantidade negativa é rejeitada
- Verificar se uma compra sem descrição é rejeitada
- Verificar se o enum de categoria aceita apenas valores válidos (CERVEJA, VINHO, etc.)

Como rodar os testes

No IntelliJ, clique com botão direito na pasta test → Run All Tests.

Ou pelo terminal dentro de cada serviço:

```
.\mvnw test
```

Parte 10 — Salvar seu trabalho no GitHub

Depois de escrever e rodar os testes, você precisa salvar tudo no GitHub na sua branch.

Passo 1 — Verificar o que mudou

```
git status
```

Vai aparecer a lista de arquivos modificados.

Passo 2 — Adicionar os arquivos

```
git add .
```

Passo 3 — Fazer o commit

```
git commit -m "testes: adiciona testes unitários do serviço de vendas"
```

Mude a mensagem conforme o que você fez.

Passo 4 — Subir para o GitHub

```
git push origin testes_SEU_NOME
```

Substitua testes_SEU_NOME pelo nome da sua branch (ex: testes-joao).

Parte 11 — Unir seu trabalho na main

Quando seus testes estiverem prontos e aprovados pelo grupo, você faz o merge na main.

Opção 1 — Pelo GitHub (recomendado)

48. Acesse o repositório: https://github.com/JoaoPedroLopesOliveira/distribuidora_de_bebidas
49. Vai aparecer um aviso amarelo com "Compare & pull request" — clique nele
50. Descreva o que você fez
51. Clique em Create pull request
52. Peça para outro integrante revisar e aprovar
53. Clique em Merge pull request

Opção 2 — Pelo terminal

```
git checkout main
git pull origin main
git merge testes_SEU_NOME
git push origin main
```

Testes funcionais com Talend API Tester

O Talend é uma extensão do Chrome para testar os endpoints da aplicação.

54. Instale a extensão: <https://chrome.google.com/webstore/detail/talend-api-tester>

55. Abra o Chrome e clique na extensão

Como usar o Talend

- Method: tipo da requisição (GET, POST, DELETE)
- URL: endereço do endpoint
- Body: dados que você envia (só para POST)
- Send: botão para enviar a requisição
- Status: código de resposta (201 = criado, 200 = ok, 404 = não encontrado)

Teste 1 — Cadastrar produto no Estoque

Campo	Valor
Method	POST
URL	http://localhost:8081/api/estoque

Body (JSON):

```
{
  "descricao": "Cerveja Heineken",
  "categoria": "CERVEJA",
  "quantidade": 200
}
```

✓ Esperado: status 201 Created e o produto aparece na resposta com um id.

Teste 2 — Listar todos os produtos do Estoque

Campo	Valor
Method	GET
URL	http://localhost:8081/api/estoque

✓ Esperado: status 200 OK e uma lista com o produto cadastrado.

Teste 3 — Registrar uma Compra

Campo	Valor
Method	POST
URL	http://localhost:8082/api/compras

Body (JSON):

```
{
  "descricao": "Cerveja Heineken",
  "categoria": "CERVEJA",
  "precoCusto": 5.50,
  "quantidadeComprada": 50,
  "idProduto": 1
}
```

✓ Esperado: status 201 Created. O estoque deve aumentar de 200 para 250.

Teste 4 — Registrar uma Venda

Campo	Valor
Method	POST
URL	http://localhost:8083/api/vendas

Body (JSON):

```
{
  "descricao": "Cerveja Heineken",
  "categoria": "CERVEJA",
  "valorUnitario": 10.00,
  "quantidadeVendida": 10,
  "idProduto": 1
}
```

✓ Esperado: status 201 Created. O estoque deve diminuir de 250 para 240.

Teste 5 — Verificar se o estoque foi atualizado pelo Kafka

Campo	Valor
Method	GET
URL	http://localhost:8081/api/estoque/1

✓ Esperado: quantidade 240 — isso confirma que o Kafka comunicou corretamente os serviços.

Teste 6 — Buscar registro que não existe

Campo	Valor
Method	GET
URL	http://localhost:8081/api/estoque/9999

✓ Esperado: status 404 Not Found.

Teste 7 — Deletar uma venda

Primeiro liste as vendas para pegar um id:

Campo	Valor
Method	GET
URL	http://localhost:8083/api/vendas

Depois delete pelo id:

Campo	Valor
Method	DELETE
URL	http://localhost:8083/api/vendas/1

✓ Esperado: status 204 No Content.

Dúvidas Frequentes

Erro: "Unknown database"

Você não criou os bancos ainda. Volte para a Parte 4 e crie os bancos no MySQL Workbench.

Erro: "Access denied"

A senha do banco está errada. Volte para a Parte 5 e ajuste a senha correta no `application.properties`.

Erro de Kafka ao subir o serviço

O Docker não está rodando. Abra o Docker Desktop e rode `docker-compose up -d` novamente.

O estoque não atualiza após venda/compra

Verifique se os três serviços estão rodando ao mesmo tempo. Todos precisam estar ativos para o Kafka funcionar.

Erro de importações no IntelliJ (tudo vermelho)

Clique com botão direito no `pom.xml` → Maven → Reload Project.

Se não resolver: File → Invalidate Caches → Invalidate and Restart.